

SmartBugs - Regression Test Execution Report

Luciano Bercini

April 2026

Contents

1	Introduction	2
2	Regression Scope	2
3	Executed Regression Test Suite	2
4	Test Environment	2
5	Regression Test Execution Results	3
5.1	Overall Results	3
5.2	Executed Test File	3
6	Validated Unchanged Functionalities	3
7	Relation with Pre-Modification Test Cases	4
8	Coverage Results	4
9	Additional Note on Broader Legacy Test Execution	4
10	Conclusion	4

1 Introduction

This document reports the execution of the regression tests performed after the implementation of the proposed modifications to SmartBugs.

The goal of regression testing is to verify that the original SmartBugs behavior has been preserved after the introduction of the intelligent tool-selection workflow. In particular, regression testing focuses on the functionalities that were already present before the modification and that must remain unchanged.

As defined in the testing documents, the regression strategy is based on the reuse of the pre-modification test cases corresponding to the original SmartBugs orchestration workflow.

2 Regression Scope

The regression execution focused on the unchanged test cases associated with the original SmartBugs workflow. These tests validate the core orchestration behavior of the system, including:

- file collection;
- task generation;
- tool and alias handling;
- handling of Solidity, bytecode, and runtime bytecode inputs;
- management of compiler-related errors;
- result directory collision handling;
- main orchestration flow.

These aspects correspond to the unchanged pre-modification test cases reused for regression testing.

3 Executed Regression Test Suite

The following regression test suite was executed:

- `tests/unit/test_smartbugs.py`

The command used to execute the regression tests was:

```
pytest tests/unit/test_smartbugs.py --cov-fail-under=0
```

4 Test Environment

The regression tests were executed in the following environment:

- Operating System: Windows
- Python version: 3.11.0
- Test framework: `pytest 9.0.3`
- Additional pytest plugins:
 - `pytest-cov 7.1.0`
 - `pytest-mock 3.15.1`
 - `pytest-timeout 2.4.0`

The execution was performed inside the project virtual environment.

5 Regression Test Execution Results

5.1 Overall Results

The regression execution completed successfully.

- Total collected tests: 37
- Passed tests: 37
- Failed tests: 0
- Errors: 0
- Skipped tests: 0

Therefore, all executed regression tests passed successfully.

5.2 Executed Test File

Test File	Type	Result
tests/unit/test_smartbugs.py	Regression / Unit / Component	Passed

Table 1: Executed regression test suite and result

6 Validated Unchanged Functionalities

The executed regression tests validated the preservation of the original SmartBugs behavior.

In particular, the following unchanged functionalities were re-validated:

- collection of Solidity files from explicit paths and glob patterns;
- collection of bytecode and runtime bytecode files;
- handling of `.sbd` dataset files;
- filtering of unsupported file types;
- task generation for valid file and tool combinations;
- handling of duplicate input files;
- task generation for multiple tools and multiple files;
- handling of tools requiring Solidity compiler resolution;
- error reporting for missing pragmas or missing matching compiler versions;
- support for the `continue_on_errors` behavior;
- handling of result directory collisions;
- Docker image loading decisions during task assembly;
- tool loading and tool alias expansion;
- preservation of the original main orchestration flow.

These results confirm that the modifications introduced for the intelligent workflow did not alter the core orchestration behavior already present in the original SmartBugs system.

7 Relation with Pre-Modification Test Cases

The executed regression suite provides implementation-level evidence for the unchanged pre-modification test cases reused after the system evolution.

In particular, the executed tests cover the original regression scope corresponding to:

- TC-PRE-01: Collection of a single valid Solidity file
- TC-PRE-02: Collection of multiple Solidity files using a glob pattern
- TC-PRE-03: Collection and task generation for a bytecode file
- TC-PRE-04: Task generation for runtime bytecode
- TC-PRE-05: Basic task generation for a Solidity file and a compatible tool
- TC-PRE-06: Task generation for one Solidity file and multiple compatible tools
- TC-PRE-07: Missing pragma with a solc-dependent tool
- TC-PRE-08: Incompatible file and tool mode
- TC-PRE-09: Loading a tool alias
- TC-PRE-10: Handling result directory collisions
- TC-PRE-11: Main SmartBugs orchestration workflow

Thus, the regression execution is consistent with the unchanged test cases identified in the post-modification test case specification.

8 Coverage Results

The regression test execution produced the following relevant coverage value for the main unchanged orchestration module:

Module	Coverage
<code>sb.smartbugs.py</code>	84.80%

Table 2: Relevant regression coverage result

9 Additional Note on Broader Legacy Test Execution

A broader exploratory execution of additional legacy unit tests was also attempted on the Windows environment.

That broader execution revealed a small number of failures related to platform-dependent path-format expectations in legacy tests, especially concerning path separators and Windows-specific path handling. These issues were not related to the newly introduced intelligent workflow.

For this reason, the formal regression evidence reported in this document is based on the regression suite aligned with the unchanged pre-modification orchestration test cases, namely `tests/unit/test_smartbugs.py`.

10 Conclusion

The regression test execution was successful.

All 37 executed regression tests passed, confirming that the original SmartBugs orchestration behavior has been preserved after the implementation of the intelligent tool-selection workflow.

The results provide evidence that the proposed evolution is backward compatible with the original manual SmartBugs workflow and does not introduce regressions in the unchanged core orchestration logic.